PROJECT DOCUMENTATION

NFT Collection on Foundry

A fully on-chain ERC-721 implementation featuring two contracts — a URI-based NFT and a dynamic mood-shifting NFT with embedded base64 SVG artwork.

Solidity ^0.8.18 • Foundry • OpenZeppelin v5 • Anvil

1. Project Overview

This project demonstrates two complementary NFT implementations using Foundry's scripting, testing, and deployment toolchain. Both contracts extend OpenZeppelin's `ERC721` and use a shared set of deployment and interaction scripts.

CONTRACT A

BasicNFT

A standard ERC-721 where the token URI is supplied at mint time and stored per token. Ideal for IPFS-hosted metadata.

CONTRACT B

MoodNFT

A dynamic ERC-721 whose artwork is generated on-chain. The owner can flip a token between two moods (HAPPY / SAD), each rendering a different base64-encoded SVG.

> **Design principle.** All metadata and imagery for `MoodNFT` live entirely on-chain as base64 data URIs — no external servers, no IPFS dependency, no metadata endpoint.

2. Architecture & Tech Stack

LAYER	TECHNOLOGY / PURPOSE
Language	Solidity `^0.8.18`
Framework	Foundry (Forge, Anvil, Cast)
Token standard	ERC-721 via OpenZeppelin Contracts v5
Encoding	OpenZeppelin `Base64` utility
Local chain	Anvil (Chain ID 31337, chain name `anvil-hardhat`)
Deployment	Solidity scripts with `vm.startBroadcast()`
Testing	Forge unit & integration tests with `vm.prank`
Helpers	ChainAccelOrg `foundry-devops` for recent deployment lookup

Directory layout

```
NFT-Collection/
├── src/
│   ├── BasicNFT.sol              // URI-based ERC-721
│   └── MoodNft.sol               // on-chain SVG ERC-721
├── script/
│   ├── DeployBasicNft.s.sol
│   ├── DeployMoodNft.s.sol
│   └── MintBasicNft.s.sol
├── test/
│   ├── Integration/
│   │   ├── BasicNftTest.t.sol
│   │   └── MoodNftIntegrationTest.t.sol
│   └── Unit/
│       ├── DeployMoodNftTest.t.sol
│       └── MoodNftTest.t.sol
├── img/
│   ├── sad.svg
```

```
│   └── happy.svg
└── foundry.toml
```

3. Smart Contracts

BasicNFT ERC-721

A minimal ERC-721 where the caller supplies an arbitrary URI string at mint time. Each token stores its own URI. Names and symbols are fixed at construction (`"BasicNFT" / "BNFT"`).

FUNCTION	BEHAVIOUR
`mintNft(string _tokenURI)`	Stores the URI for the next sequential token ID and mints it to `msg.sender`.
`tokenURI(uint256)`	Returns the stored URI for the given token.

> **Note.** `BasicNFT` does not validate that the token exists — `tokenURI` returns an empty string for unminted IDs rather than reverting. Acceptable for this exercise; use `_requireOwned(tokenId)` if stricter semantics are needed.

MoodNFT ERC-721 · ON-CHAIN SVG

A dynamic NFT storing one of two moods per token. `tokenURI` returns a fully-formed `data:application/json;base64,…` URI whose embedded `image` field is itself a `…` payload. No external host is ever contacted.

FUNCTION	BEHAVIOUR
`mintNft()`	Mints the next token to `msg.sender` and initialises it as `HAPPY`.
`flipMood(uint256)`	Toggles HAPPY ↔ SAD. Reverts with `MoodNft_CantFlipMoodIfNotOwner` if caller is not the token owner.
`getMood(uint256)`	Returns `true` for HAPPY, `false` for SAD.
`tokenURI(uint256)`	Selects the correct SVG URI based on mood and returns a base64 JSON data URI.

Metadata schema

```
{
  "name": "Mood NFT",
  "description": "An NFT that changes based on the mood.",
  "attributes": [{ "trait_type": "moodiness", "value": 100 }],
  "image": "…"
}
```

4. Deployment & Interaction Scripts

SCRIPT	PURPOSE
`DeployBasicNft.s.sol`	Deploys `BasicNFT`. Exposes `deployBasicNft()` returning the instance for test reuse.
`DeployMoodNft.s.sol`	Reads `img/sad.svg` and `img/happy.svg`, converts both to base64 image URIs, and deploys `MoodNFT`. Exposes `svgToImageURI()` for reuse in tests.
`MintBasicNft.s.sol`	Resolves the most recent `BasicNFT` deployment using `DevOpsTools` and mints a token to it.

> **Broadcast pattern.** Each script wraps only the deployment (or minting) call inside `vm.startBroadcast()` and `vm.stopBroadcast()`, with the `stopBroadcast()` call placed *before* the `return` statement. This is what allows the scripts to be safely reused inside test `setUp()` functions without leaking a broadcast session into the test body.

5. Test Suite

Tests are split into **unit** (single contract, isolated) and **integration** (deployment script + contract interaction). All tests use `vm.prank()` to simulate the caller and `makeAddr()` to produce deterministic EOAs.

TEST FILE	COVERAGE
`test/Integration/BasicNftTest.t.sol`	Deployment name check · mint via `vm.prank(user)` · balance and URI round-trip
`test/Integration/MoodNftIntegrationTest.t.sol`	tokenURI rendering after mint · owner-only flip reverting and succeeding
`test/Unit/DeployMoodNftTest.t.sol`	Deployment success · `svgToImageURI` encoding correctness
`test/Unit/MoodNftTest.t.sol`	Direct mint + `tokenURI` output logging

Latest test results

```
BasicNftTest::testNameIsCorrect                    PASS
```

`BasicNftTest::testCanMintAndHaveBalance`	PASS
`MoodNftTest::testViewTokenURI`	PASS
`MoodNftTest::testViewTokenURIIntegration`	PASS
`MoodNftTest::testFlipTokenToSad`	PASS
`DeployMoodNftTest::testCanDeployMoodNft`	PASS
`DeployMoodNftTest::testConvertSvgToImageURI`	PASS

> **Result.** 6 test suites — 100% passing. The `svgToImageURI` unit test verifies byte-for-byte equality between the expected base64 string and the generated one, guarding against any accidental change to the encoding pipeline.

6. Local Deployment Workflow

1 · Start Anvil

```
anvil
```

Launches a local chain on `http://127.0.0.1:8545` with ten pre-funded accounts. Note the private key of account #0 for deployments.

2 · Deploy a contract

```
forge script script/DeployMoodNft.s.sol:DeployMoodNft \
  --rpc-url http://127.0.0.1:8545 \
  --broadcast \
  --interactive
```

The `--interactive` flag prompts for the private key without writing it to shell history. Example deployed address observed during this project:

`0x5FbDB2315678afecb367f032d93F642f64180aa3` at block 1.

3 · Interact with the deployed contract

```
# Mint
cast send <ADDRESS> "mintNft()" \
  --rpc-url http://127.0.0.1:8545 \
  --private-key <KEY>

# Read tokenURI
cast call <ADDRESS> "tokenURI(uint256)" 0 \
  --rpc-url http://127.0.0.1:8545

# Read mood
cast call <ADDRESS> "getMood(uint256)" 0 \
  --rpc-url http://127.0.0.1:8545
```

7. Key Concepts Demonstrated

ON-CHAIN ARTWORK

Base64 data URIs

SVG images are encoded into base64 and embedded directly inside the `tokenURI` JSON, eliminating external hosting.

ACCESS CONTROL

Owner-only state change

`flipMood` compares `msg.sender` against `ownerOf(tokenId)` and reverts with a custom error.

SCRIPT REUSE

Returning helper functions

Deployment scripts expose `deploy*()` helpers that return the instance, so tests can call them from `setUp()` without duplicating deploy logic.

CHEATCODE SAFETY

Broadcast + prank isolation

Every broadcast is closed *before* returning, so `vm.prank()` in tests never trips over an open broadcast session.

8. Configuration Notes

`foundry.toml` includes the following non-default entries:

```
[profile.default]
src = "src"
out = "out"
libs = ["lib"]
remappings = [
  "@openzeppelin/contracts/=lib/openzeppelin-contracts/contracts/",
  "forge-std/=lib/forge-std/src/"
]
fs_permissions = [{ access = "read", path = "./img/" }]
```

- **Remappings** — `@openzeppelin/contracts/` matches the import paths used throughout OpenZeppelin's documentation.
- **fs_permissions** — required for `vm.readFile("./img/sad.svg")` and `happy.svg` inside `DeployMoodNft.s.sol` and its test.

9. Known Limitations & Future Work

- **Import style is mixed.** Some files use remapped `forge-std/…` paths while others use hardcoded `../lib/forge-std/src/…`. Both resolve locally; only the remapped style is portable across machines.
- **Case-sensitive filesystems.** A few historical imports referenced `MoodNFT.sol` (capital `NFT`) while the file on disk is `MoodNft.sol`. This resolves on macOS but would break on Linux-based CI.
- **`BasicNFT` token existence.** `tokenURI` returns an empty string for unminted IDs rather than reverting. Adding `_requireOwned(tokenId)` would tighten this.
- **Duplicate deployment logic.** `DeployMoodNft` contains both a `run()` and a `deployMoodNft()` that read the same files and call the same constructor. This is intentional for test reuse, but the two could be collapsed into one helper.
- **No metadata versioning.** Any future change to the JSON schema (name, description, attributes) would alter the `tokenURI` output. There is no on-chain version marker.